भारत अंतरिक्ष अकादमी, अंतरिक्ष शिक्षा विभाग
India Space Academy, Department of Space Education
भारत अंतरिक्ष सप्ताह / India Space Week

Development of an AI Based Real Time Anomaly Detection and Autonomous Decision Support System for Deep Space Missions

Artificial Intelligence and Machine Learning in Space Exploration

---

Project Report

Submitted By

Purushothaman S

M25DE1033 | MTech. — Data Engineering

Indian Institute of Technology Jodhpur

m25de1033@iitj.ac.in | push.san25@gmail.com

India Space Academy | 2025 – 2026

Git Hub: Click here to get the project from the Git

Contents

1. Abstract 4
2. Introduction 5
   2.1 The Communication Delay Problem 5
   2.2 Project Objectives 5
   2.3 Relevance to Real Missions 5
3. Dataset 6
   3.1 NASA CMAPSS FD001 Dataset 6
   3.2 Why This Dataset Was Selected 6
      Structural similarity to spacecraft subsystem telemetry 6
      Progressive run to failure data with known ground truth 6
      Multivariate sensor interactions 6
      Real world provenance and benchmark comparability 7
4. Project Structure 7
   4.1 Directory Layout 7
   4.2 Docker Services 8
5. Implementation Architecture 9
   5.1 Four Layer Design 9
   5.2 End to End Data Flow 9
6. Schematic Diagram of the Architecture 10
7. Detailed Explanation of Each Block 12
   7.1 Data Loader (src/data_loader.py) 12
   7.2 Preprocessor (src/preprocess.py) 12
   7.3 Feature Engineering (src/feature_engineering.py) 12
   7.4 Isolation Forest Anomaly Detector (src/train_anomaly.py) 12
   7.5 Autoencoder Deep Learning Anomaly Detector (src/train_autoencoder.py) 13

7.6 Random Forest RUL Predictor (src/train_rul.py) 13
7.7 LSTM Deep Learning RUL Predictor (src/train_rul_lstm.py) 13
7.8 Rule Based Decision Engine (src/decision_engine.py) 14
7.9 Risk Scoring and Confidence Estimation (src/decision_confidence.py) 14
7.10 Reinforcement Learning Agent (src/decision_confidence.py) 14
7.11 FastAPI Inference Server (api/) 15
7.12 Kafka Streaming Pipeline (kafka/) 15
7.13 Streamlit Mission Control Dashboard (app/streamlit_app.py) 16
7.14 Live Stream Dashboard Integration (api/main.py + app/streamlit_app.py) 16
7.15 Prometheus and Grafana Monitoring (monitoring/) 17
8. Step by Step Grafana Dashboard Setup 17
8.1 Add Prometheus as a Data Source 18
8.2 Create a New Dashboard and Add Panels 18
8.3 Arrange and Save 20
9. Advantages of This Project 20
9.1 Fully Autonomous Operation with No Human Dependency 20
9.2 Multi Model Redundancy 20
9.3 Transparent and Fully Explainable Decisions 20
9.4 Self Improving Policy through Reinforcement Learning 21
9.5 Production Grade Containerised Deployment 21
9.6 Real Time Operational Self Monitoring 21
9.7 Real World Validated on an Established Benchmark 21
9.8 Streaming Architecture Ready for Live Telemetry 21
9.9 Live Dashboard Integration with Kafka Streaming 22
10. Conclusion 23

Abstract

Deep space missions such as India's Chandrayaan-3 and Aditya-L1 face a challenge that cannot be resolved by faster communication technology alone the physical delay imposed by the speed of light. When a spacecraft approaches Mars, a radio signal from Earth takes between three and twenty two minutes to arrive, and another three to twenty two minutes to return. By the time a ground team receives an alarm, decides on a response, and transmits a command, an unfolding failure may already have caused irreversible damage.

This project responds to that challenge by building a fully autonomous AI system that monitors spacecraft telemetry, detects anomalies, predicts how many operational cycles remain before a component fails, and makes corrective decisions entirely on its own, without any contact with Earth. The system is not a proof of concept built around a single model. It layers two complementary anomaly detectors, two independent RUL predictors, a three tier decision system that moves from explicit rules to probabilistic risk scoring to a self improving reinforcement learning agent, and a complete operational deployment stack.

Every recommendation the system produces is accompanied by a plain English justification that names the exact evidence behind the decision. This transparency is critical for aerospace applications where engineers need to understand and trust the AI's behaviour. The system is validated using the NASA CMAPSS Turbofan Engine Degradation Dataset, a real world aerospace benchmark whose degradation patterns closely mirror those of spacecraft subsystems. It is deployed as eight Docker containers that start with a single command, providing a live dashboard, a REST API, real time streaming, and operational monitoring.

A key feature of the deployed system is the real time integration between the Kafka streaming pipeline and the Streamlit dashboard. Every prediction made by the FastAPI inference server triggered by the Kafka consumer is stored in an in memory live prediction store and immediately visible on the dashboard's Live Stream page, which polls the API every three seconds and updates all charts, gauges, and the prediction feed in real time.

Introduction

2.1 The Communication Delay Problem

Every spacecraft is a collection of machines working together in an environment where no engineer can physically reach them. On Earth, when a machine behaves unusually, a technician can investigate immediately. In space, the only option is to send a command from Earth but the speed of light imposes a hard limit on how quickly that command can arrive.

Mission	Distance from Earth	One Way Signal Delay
International Space Station	Approximately 400 km	Under 0.001 seconds
Moon — Chandrayaan-3	Approximately 384,000 km	About 1.3 seconds
Sun-Earth L1 — Aditya-L1	About 1.5 million km	About 5 seconds
Mars — Perseverance Rover	56 million to 401 million km	3 to 22 minutes

A roundtrip command to Mars can consume 44 minutes. If a component is about to fail and the spacecraft needs to shut down a subsystem, it cannot wait 44 minutes for instructions from the ground. The spacecraft must reason and decide on its own. This is the core engineering challenge that this project was built to address, and solving it requires an on board AI system capable of the same diagnostic reasoning that a skilled human engineer would apply.

2.2 Project Objectives

The project was designed to demonstrate that a complete autonomous health management system is achievable using current machine learning techniques combined with modern deployment infrastructure. The specific objectives are to build anomaly detection models that identify unusual sensor behaviour in real time, implement time series forecasting to predict how many operational cycles remain before failure, develop an AI based decision engine that recommends corrective actions without human input, create a live mission control dashboard for visual monitoring, and deploy the entire system using Docker so it runs on any machine with a single command.

2.3 Relevance to Real Missions

Although this project uses simulated engine telemetry for validation, its architecture transfers directly to real spacecraft subsystems. Reaction wheels, power converters, thermal control units, and propulsion systems all degrade in the same gradual way as turbofan engines sensor readings drift slowly at first and then accelerate toward failure. Replacing the NASA dataset with actual spacecraft telemetry and calibrating the thresholds to the specific failure modes of a real mission would require minimal changes to the system's core architecture.

Dataset

3.1 NASA CMAPSS FD001 Dataset

The project uses the NASA CMAPSS Turbofan Engine Degradation Dataset, specifically the FD001 training subset. This dataset was produced by NASA's Prognostics Center of Excellence and is one of the most widely referenced benchmarks in predictive maintenance and remaining useful life research.

Property	Details
Dataset name	NASA CMAPSS FD001
Number of engines	100 simulated turbofan engines
Sensor measurements	21 sensor readings per engine per cycle
Operational settings	3 operating conditions recorded per cycle
Total input columns	26 columns: unit ID, time cycle, 3 settings, 21 sensors
Total observations	20,631 rows across all engines and cycles
Cycles per engine	Typically 128 to 362 cycles before failure
Operating condition	Single condition — HPC degradation mode
File format	Whitespace separated text with no column headers

3.2 Why This Dataset Was Selected

The CMAPSS dataset was selected for reasons that directly support the project's goal of building a system applicable to real deep space missions.

Structural similarity to spacecraft subsystem telemetry

Spacecraft subsystems degrade in the same way turbofan engines do. Sensor readings drift slowly during early operation and then accelerate toward failure. The gradual, multi sensor degradation captured in CMAPSS is representative of what engineers observe in spacecraft reaction wheels, power electronics, and thermal systems.

Progressive run to failure data with known ground truth

The dataset records each engine from its very first cycle through to the cycle of failure. This means the true Remaining Useful Life is known at every point in time, which allows the RUL prediction models to be trained with precise labels and evaluated against verifiable ground truth a property that is rare and highly valuable in predictive maintenance research.

Multivariate sensor interactions

With 21 sensors capturing different physical properties simultaneously, the dataset demands models that understand how multiple signals relate to each other. A temperature increase is more significant when accompanied by a pressure drop. This multivariate complexity is exactly what makes CMAPSS a realistic proxy for the multi sensor environment of a spacecraft.

Real world provenance and benchmark comparability

The dataset was generated using a high fidelity physics simulation developed by NASA, making it far more representative of real hardware behaviour than purely synthetic alternatives. Because CMAPSS is a standard benchmark, results produced on it can be compared directly against published research, providing independent validation that the models are performing at a competitive level.

Project Structure

4.1 Directory Layout

The project is organised so that every component has a single, clearly defined responsibility. The structure separates data, machine learning logic, API, streaming, monitoring, and deployment concerns into distinct folders.

```
space-telemetry-anomaly-system/
|
+-- main.py                      Single entry point: runs the full ML pipeline
+-- requirements.txt             All Python dependencies
+-- Dockerfile                   Streamlit dashboard container image
+-- Dockerfile.api               FastAPI inference server container image
+-- Dockerfile.pipeline          ML training container (includes TensorFlow)
+-- Dockerfile.consumer          Kafka consumer container image
+-- docker-compose.yml           Starts all 8 services with one command
+-- entrypoint.sh                Dashboard startup script and URL banner printer
|
+-- src/
|   +-- config.py                Central configuration: paths and
hyperparameters
|   +-- data_loader.py           Loads NASA C-MAPSS text file into a DataFrame
|   +-- preprocess.py            Calculates RUL, applies StandardScaler
|   +-- feature_engineering.py   Rolling means and cycle-to-cycle differences
|   +-- train_anomaly.py         Isolation Forest training and inference
|   +-- train_autoencoder.py     Deep learning Autoencoder training
|   +-- train_rul.py             Random Forest RUL model training
|   +-- train_rul_lstm.py        LSTM sequence model training
|   +-- decision_engine.py       Rule-based autonomous decision logic
|   +-- decision_confidence.py   Risk scoring, confidence estimation, RL agent
|   +-- evaluate.py              Confusion matrix and classification report
|   +-- simulator.py             Telemetry stream simulator
|   +-- utils.py                 Shared utility functions
|
+-- app/
|   +-- streamlit_app.py         6-page mission-control dashboard (incl. Live
Stream)
|
+-- api/
|   +-- main.py                  FastAPI server with /predict, /live, /metrics
|   +-- inference.py             Model loading and prediction logic
```

```
|   +-- schemas.py              Pydantic request and response data models
|   +-- metrics.py              Prometheus counters and latency histograms
|
+-- kafka/
|   +-- producer.py             Publishes telemetry rows to Kafka topic
|   +-- consumer.py             Reads Kafka topic and calls /predict API
|   +-- stream_pipeline.py      Combined producer and consumer pipeline
|
+-- monitoring/
|   +-- prometheus.yml          Prometheus scrape target configuration
|   +-- grafana/                Grafana provisioning and dashboard files
|
+-- data/raw/train_FD001.txt    NASA C-MAPSS raw training data
+-- models/                     Saved trained model files (.pkl and .keras)
+-- outputs/                    Results CSV, evaluation metrics, figures, logs
+-- tests/                      Automated test suite using pytest
```

4.2 Docker Services

All eight services are defined in docker compose.yml and start automatically in the correct dependency order when you run docker compose up --build.

Container	Purpose	Port
telemetry-pipeline	Trains all ML models, writes outputs and saved models	None
telemetry-api	FastAPI server — /predict, /live, /metrics endpoints	8000
telemetry-dashboard	Streamlit dashboard with 6 pages including Live Stream	8501
telemetry-kafka-consumer	Streams telemetry through Kafka to the API	None
telemetry-zookeeper	Coordination service required by Kafka	2181
telemetry-kafka	Apache Kafka message broker	9092
telemetry-prometheus	Scrapes API metrics every 5 seconds	9090
telemetry-grafana	Operational monitoring and alerting dashboards	3000

Implementation Architecture

Four Layer Design

The system is structured in four layers, each with a distinct responsibility. Data enters at the bottom and is progressively enriched as it moves upward toward the decision and delivery layers.

Layer	Responsibility
Layer 1 — Data	Loads raw sensor data, calculates RUL, engineers features, scales values.
Layer 2 — Intelligence	Runs anomaly detection and RUL prediction models to produce scores and forecasts.
Layer 3 — Decision	Combines model outputs into recommendations, risk scores, confidence, and justifications.
Layer 4 — Delivery	Serves results via REST API, streams them through Kafka, visualises them on a live dashboard, tracks metrics in Prometheus and Grafana.

5.2 End to End Data Flow

The table below traces the journey of a single sensor reading from raw input all the way to a displayed recommendation on the mission control dashboard.

Step	What Happens
1. Raw input	A row of 26 values arrives: engine ID, cycle number, 3 operational settings, and 21 sensor readings.
2. RUL calculation	Remaining useful life is computed as maximum cycle minus current cycle for that engine.
3. Feature engineering	Rolling averages and cycle differences are computed for six key sensors to capture trends over time.
4. Scaling	All features are normalised to zero mean and unit variance using a fitted StandardScaler.
5. Isolation Forest	Checks whether the pattern looks like a normal engine. Produces anomaly_flag and anomaly_score.
6. Autoencoder	Reconstructs the input from a compressed representation. High reconstruction error signals an anomaly.
7. Random Forest RUL	Predicts remaining cycles using the current feature snapshot as a standalone observation.
8. LSTM RUL	Predicts remaining cycles using the last 30 cycles as a temporal sequence.
9. Decision engine	Combines anomaly flag, anomaly score, and predicted RUL to issue NORMAL / CAUTION / WARNING / CRITICAL.
10. Risk and confidence	Calculates a weighted risk score (0 to 1) and assesses whether the two signals agree.
11. Justification	Writes a plain English sentence naming the specific values that drove the recommendation.

12. Live store + delivery	Result is saved to LIVE_PREDICTIONS in memory store, served via /predict and /live API, streamed via Kafka, shown live on dashboard, tracked in Prometheus.

Schematic Diagram of the Architecture

The diagram below shows how the four layers of the system connect and how data flows from raw sensor input through intelligence and decision making to final delivery. Read the diagram from top to bottom, following the data arrows.

```
              +---------------------------------+
              |    NASA C-MAPSS SENSOR DATA      |
              |  21 sensors  x  N cycles         |
              +----------------+-----------------+
                               |
            ==================|===================
            LAYER 1            |    DATA LAYER
            ==================|===================
      +----------+   +---------+--------+   +--------------+
      | Data     +-->+ RUL Calculation  +-->+ Feature Eng. |
      | Loader   |   | max_cyc - cyc    |   | Roll mean    |
      +----------+   +------------------+   | Cycle diff   |
                                             +------+-------+
                                                    |
                                             StandardScaler
                                                    |
            ==================|===================
            LAYER 2            |    INTELLIGENCE LAYER
            ==================|===================
      +------------------------+------------------------+
      |   ANOMALY DETECTION    |   RUL PREDICTION       |
      +-----------+------------+-----------+------------+
      | Isolation | Autoencoder| Random    | LSTM       |
      | Forest    | Dense NN   | Forest    | Sequence   |
      +-----+-----+------+-----+------+----+-----+------+
            |            |             |         |
      anomaly_flag  recon_error    pred_RUL  pred_RUL
      anomaly_score                (baseline) (deep)
            |            |             |         |
            +-----+------+             +----+----+
                  |                        |
             =====|========================|=======
             LAYER 3       DECISION LAYER
             =====|========================|=======
      +-----------+-------------------------+-----------+
      |           RULE-BASED DECISION ENGINE            |
```

```
        |  IF anomaly AND RUL<=5   -->  CRITICAL         |
        |  IF anomaly AND RUL<=15  -->  HIGH RISK        |
        |  IF anomaly              -->  WARNING           |
        |  IF RUL<=10              -->  CAUTION           |
        |  ELSE                    -->  NORMAL            |
        +-------------------+-------------------------------+
                            |
        +-------------------+------------------+
        | Risk Scorer                           |
        | 0.60 x RUL_risk + 0.40 x AnomalyRisk|
        +-------------------+------------------+
                            |
        +-------------------+-------------------+
        | Confidence Estimator                   |
        | HIGH / MEDIUM / LOW                    |
        +-------------------+-------------------+
                            |
        +-------------------+-----------------+
        | Justification Builder               |
        | Plain-English explanation           |
        +-----------+-------------------------+
                    |
        +-----------+-----------+
        | RL Agent (Q-learning) |
        | Learns optimal policy |
        +-----------+-----------+
                    |
               =====|==================================
               LAYER 4         DELIVERY LAYER
               =====|==================================
                    |
     +--------------+-----------+-----------+----------+
     |              |           |           |          |
+---+----+   +-----+---+   +--+-----+  +--+-----+  +-+-------+
| FastAPI|   | Kafka   |   |Streamlit|  |Prometh.|  | Grafana |
| /predict   |Producer |   |Dashboard|  |Metrics |  | Panels  |
| /live  |   |Consumer |   |6 pages  |  |Scrape  |  | Alerts  |
| /metrics   +---------+   +----+----+  +--------+  +---------+
+----+---+                      |
      |              +-----------+
      |              | Live Stream Page polls GET /live
      |              | every 3 seconds -- updates charts
      |              | KPIs, anomaly feed in real time
      +--------------+
  LIVE_PREDICTIONS deque (max 500)
  stores every /predict result
  with timestamp + latency_ms
```

```
Port 8000       Port 9092     Port 8501    Port 9090   Port 3000
```

Detailed Explanation of Each Block

7.1 Data Loader (src/data_loader.py)

The data loader is the entry point of the pipeline. Its purpose is to read the raw NASA CMAPSS text file from disk and convert it into a structured table that every downstream component can work with reliably. The raw file has no column headers and is simply rows of space separated numbers. The loader assigns the correct column names: unit_id identifies which engine each row belongs to, time_cycle records which operational cycle it is, followed by three operational settings and twenty one sensor readings. It handles edge cases such as extra empty columns that appear in some file versions and raises a clear, descriptive error if the file is missing or has an unexpected number of columns, preventing silent data corruption from propagating through the rest of the pipeline.

7.2 Preprocessor (src/preprocess.py)

The preprocessor performs two essential transformations. The first calculates the Remaining Useful Life. Because the dataset records each engine from its very first cycle through to the cycle of failure, the maximum cycle for any given engine is its failure point. RUL at any cycle is that maximum minus the current cycle number. An engine that lasted 192 cycles has a RUL of 92 at cycle 100, a RUL of 1 at cycle 191, and a RUL of 0 at its final recorded cycle. The second transformation applies feature scaling using StandardScaler. Raw sensor values span vastly different numerical ranges: sensor 2 typically reads around 640 while sensor 14 reads around 8. Without scaling, a machine learning model would treat large valued sensors as more important by default, producing distorted results. Scaling normalises every feature to zero mean and unit variance, ensuring that all inputs are treated equally and the models learn only from the patterns in the data rather than from arbitrary differences in magnitude.

7.3 Feature Engineering (src/feature_engineering.py)

A single sensor reading at one point in time tells you the current state of a machine. A trend in that reading across the last twenty cycles tells you where the machine is heading. Feature engineering creates new columns that capture these trends, making the models substantially more predictive than they would be from raw readings alone. For six key sensors identified as the most informative for degradation sensors 2, 3, 4, 11, 15, and 21 two new features are computed. The rolling mean averages the last five readings for that sensor on that specific engine, smoothing out short term noise to reveal the underlying trend direction. The cycle difference records how much the sensor value changed since the previous cycle, capturing sudden spikes or accelerating drifts that may precede a fault. These engineered features transform a static snapshot into a dynamic representation that contains temporal memory about how the machine has been behaving across recent history.

7.4 Isolation Forest Anomaly Detector (src/train_anomaly.py)

The Isolation Forest is the primary anomaly detection model. It operates on an intuitive but mathematically powerful principle: normal data points resemble many others and are therefore hard

to isolate, while anomalous points are distinctive and can be separated from the crowd with very few decisions. The algorithm constructs one hundred random decision trees, each of which partitions the data by randomly selecting a feature and a split value. For a normal reading, many splits are needed before it stands alone because it is surrounded by similar readings. For an anomalous reading, very few splits suffice because it is already far from the norm. This isolation depth is converted to a score between negative one and positive one. Any reading with a score below zero is flagged as anomalous. The contamination parameter is set to five percent, reflecting the expectation that roughly one in twenty sensor readings will exhibit anomalous characteristics during the degradation phase of the engine's life.

7.5 Autoencoder Deep Learning Anomaly Detector (src/train_autoencoder.py)

The Autoencoder provides a second, independent anomaly detection capability that catches subtle multi sensor correlations the Isolation Forest might miss. It is a neural network that learns to compress sensor data into a small internal representation and then reconstruct it back to the original. The critical aspect is that it is trained exclusively on healthy engine data readings from engines with more than fifty cycles remaining. Having seen only normal operating patterns during training, it becomes very good at reconstructing normal data. When it encounters anomalous sensor readings that differ from anything in its training experience, its reconstruction is poor and the error is large. This reconstruction error is compared against a threshold set at the ninety fifth percentile of errors computed on the training data during a validation pass. Any reading that causes an error above this threshold is flagged as an anomaly. The network architecture progresses from the full feature dimension down through layers of 32, 16, and 8 units at the bottleneck, and then expands symmetrically back to the original dimension.

7.6 Random Forest RUL Predictor (src/train_rul.py)

The Random Forest is the baseline model for predicting Remaining Useful Life. It treats each row of sensor data independently, without reference to the sequence of readings that came before it, and produces a single numerical prediction: how many cycles remain before failure. The model builds one hundred decision trees, each trained on a different random sample of the training data using a random subset of input features at each branch point. This controlled randomness prevents any individual tree from memorising the training data and ensures the ensemble generalises well to engines it has never seen. The final prediction is the mean of all one hundred trees. The dataset is split eighty-twenty, with eighty percent used to train the model and twenty percent held back for performance measurement on unseen data. The trained model is saved to disk as a joblib file and loaded at startup by the FastAPI server for real time inference without retraining.

7.7 LSTM Deep Learning RUL Predictor (src/train_rul_lstm.py)

The LSTM improves on the Random Forest by treating sensor readings as a sequence rather than isolated snapshots. Instead of seeing one cycle at a time, it receives the last thirty cycles as a time window and uses the entire sequence to make its prediction. This capability matters because degradation is a trajectory, not a point. Two engines that read identical sensor values at cycle 150 are in fundamentally different situations if one has shown a steadily increasing temperature over thirty cycles while the other has remained flat. The LSTM's internal memory cells can retain information across many time steps, allowing it to learn these trajectories. The architecture consists of a single

LSTM layer with sixty four recurrent units followed by a dropout layer to reduce overfitting and a dense output layer that produces the scalar RUL prediction. Training runs for thirty epochs with batch size sixty four using the Adam optimiser and mean squared error loss. The model requires TensorFlow and is saved in the Keras .keras format.

7.8 Rule Based Decision Engine (src/decision_engine.py)

The decision engine is the autonomous control centre of the system. It receives the outputs from both anomaly detectors and both RUL predictors and translates them into a single, actionable maintenance recommendation with no human involvement. The logic consists of a priority ordered set of conditions evaluated from most critical to least. If the anomaly detector has flagged an abnormality and the predicted RUL is five cycles or fewer, the engine issues a CRITICAL alert calling for immediate maintenance. If RUL is between five and fifteen cycles with an anomaly present, it recommends an urgent inspection. A strong anomaly with more cycles remaining triggers a WARNING. A mild anomaly triggers a monitoring advisory. If no anomaly is detected but RUL has fallen below ten cycles, a CAUTION is issued to alert engineers that failure may be approaching even without visible sensor irregularity. If none of these conditions apply, the system reports nominal operations.

7.9 Risk Scoring and Confidence Estimation (src/decision_confidence.py)

Risk scoring adds a quantitative layer to the categorical recommendations produced by the decision engine. The risk score is a single number between zero and one combining two independent signals with a fixed weight. Sixty percent of the score comes from RUL proximity, which rises linearly from zero at eighty or more cycles remaining to one at zero cycles. Forty percent comes from anomaly signal strength, scaled so that a strong anomaly with a score near negative 0.5 contributes proportionally more than a borderline anomaly near zero. A confidence level then assesses whether the two signals point in the same direction. HIGH confidence is assigned when both signals agree strongly, either both indicating danger or both confirming safe operation. MEDIUM confidence applies when one signal is clearly elevated but the other is ambiguous. LOW confidence flags contradictory cases for example, a strong anomaly flag but a high remaining life which may indicate a sensor fault rather than genuine degradation. Every decision also produces a one sentence plain English justification that names the specific measurements driving the recommendation, making the AI's reasoning fully auditable.

7.10 Reinforcement Learning Agent (src/decision_confidence.py)

The reinforcement learning agent represents the most adaptive tier of the decision system. While the rule based engine follows logic programmed in advance, the RL agent discovers effective policies through experience. It observes the current state of the spacecraft expressed as a combination of anomaly presence and which of four RUL bands the engine falls into and selects one of four actions: do nothing, monitor closely, schedule an inspection, or trigger an emergency shutdown. After each action it receives a reward signal. Correct emergency shutdowns at critical RUL earn the highest reward of plus ten. Appropriate preventive inspections earn plus five. Unnecessary shutdowns when the engine is safe incur a penalty of minus five. Ignoring an anomaly incurs minus three, and failing to act at all during a critical state incurs the largest penalty of minus ten. The algorithm used is tabular Q-learning with epsilon greedy exploration. Over hundreds of training episodes, the agent shifts from random exploration toward a stable policy that consistently selects the highest reward action in every

state. The resulting Q-table demonstrates that the learned policy matches what an experienced engineer would recommend and this policy was discovered entirely through interaction with a simulated environment, not through explicit programming.

7.11 Fast API Inference Server (api/)

The FastAPI server exposes the trained models as a web service so that any connected system can request a prediction by sending an HTTP request. The /predict endpoint accepts a JSON payload containing sensor values for one engine at one cycle, runs the data through the loaded Isolation Forest and Random Forest models, applies the decision engine and confidence estimation, and returns the complete decision package: anomaly flag, anomaly score, predicted RUL, recommended action, risk score, confidence level, and justification.

In addition to inference, every result returned by /predict is immediately stored in an in memory deque called LIVE_PREDICTIONS, which holds the last 500 predictions. The /live endpoint serves this store as a JSON response, allowing the dashboard to retrieve recent predictions without any file I/O. A /live/reset endpoint is provided to clear the store and begin a fresh streaming session. The /metrics endpoint exposes all operational statistics in the Prometheus text exposition format, including total prediction requests, failure counts, anomaly detections, and a full latency histogram.

Endpoint	Description
POST /predict	Accepts sensor values, runs AI models, returns full decision. Also appends result to LIVE_PREDICTIONS with timestamp and latency.
GET /live	Returns the last N predictions from the in memory store. Polled by the dashboard every 3 seconds to power the Live Stream page.
DELETE /live/reset	Clears the LIVE_PREDICTIONS store. Use to start a fresh streaming session without restarting the API.
GET /metrics	Exposes Prometheus metrics: request counts, failure counts, anomaly counts, latency histogram.
GET /health	Simple health check endpoint confirming the API is running.

7.12 Kafka Streaming Pipeline (kafka/)

Apache Kafka provides a real time message streaming backbone that makes the system architecturally ready for live telemetry integration. The Kafka producer reads rows from the results CSV file and publishes each row as a message to a topic called telemetry input, pausing 0.3 seconds between publications to mimic a sensor stream arriving at approximately three readings per second. The consumer subscribes to this topic and processes each message as it arrives, forwarding it immediately to the FastAPI /predict endpoint and logging the response. Because /predict now saves every result to LIVE_PREDICTIONS, each Kafka driven prediction is automatically reflected on the dashboard's Live Stream page within three seconds of the API call returning. Kafka provides message durability and replay capability: if the consumer fails and is restarted, it resumes exactly where it left off without losing any messages.

7.13 Streamlit Mission Control Dashboard (app/streamlit_app.py)

The Streamlit dashboard is the human facing interface of the system, providing interactive visualisation of every model output across six pages. The first five pages present static analysis of the full training run fleet level anomaly statistics, RUL prediction charts, risk score timelines, autoencoder analysis, and an interactive reinforcement learning demo. The sixth page is the Live Stream page, which is the real time integration point between the Kafka pipeline and the dashboard.

Page	Contents
1. Overview & Anomaly Detection	Fleet KPI cards, health gauges, animated status badge, anomaly score timeline, action distribution chart.
2. RUL Prediction & Model Compare	Actual vs predicted RUL chart, prediction error over time, RF vs LSTM metrics side by side, scatter across all units.
3. Risk Score & Decision Confidence	Risk gauge, confidence level, justification card, risk score timeline with threshold lines, confidence distribution.
4. Autoencoder Analysis	Reconstruction error chart, Isolation Forest vs Autoencoder comparison, model agreement analysis, evaluation report.
5. Reinforcement Learning Demo	Interactive Q-table trainer with sliders, learned policy table, Q-value heatmap coloured by reward.
6. Live Stream	Polls GET /live every 2–10 seconds. Displays: status bar, 5 KPI cards, anomaly score timeline, predicted RUL line chart, risk area chart, recommended action bar chart, and live prediction feed table (last 20 rows, newest first).

7.14 Live Stream Dashboard Integration (api/main.py + app/streamlit_app.py)

The Live Stream page is the component that connects the Kafka streaming pipeline directly to the dashboard in real time. It works through a simple polling mechanism: the dashboard sends an HTTP GET request to the /live endpoint every few seconds, receives the latest predictions as JSON, converts them to a DataFrame, and re-renders all charts and the prediction table. This approach requires no WebSocket connections, no shared databases, and no additional infrastructure just the existing FastAPI server that is already running.

The complete real time flow works as follows. The Kafka producer reads a row from the CSV and sends it to the Kafka broker. The Kafka consumer receives the message and forwards it to POST /predict. The FastAPI server runs the AI models, generates the complete prediction result, and appends it to the LIVE_PREDICTIONS deque along with a UTC timestamp and latency measurement in milliseconds. The dashboard's Live Stream page, running in its polling loop, calls GET /live, receives the updated list, and re-renders the anomaly score chart, the predicted RUL line, the risk area chart, the recommended action bar chart, and the live feed table all in under one second. If an anomaly is detected in the latest prediction, the status bar turns red and displays the CRITICAL or WARNING action immediately.

```
Kafka Producer (reads CSV row by row, 0.3s delay)
        |
        v
Kafka Broker  (topic: telemetry input)
```

```
        |
        v
Kafka Consumer (receives message)
        |
        v
POST /predict  (FastAPI runs AI models)
        |
        +-- Returns prediction JSON to consumer (logged to terminal)
        |
        +-- Appends to LIVE_PREDICTIONS deque
               { unit_id, time_cycle, anomaly_flag, anomaly_score,
                 predicted_rul, risk_score, confidence,
                 recommended_action, justification,
                 timestamp (UTC), latency_ms }
                     |
                     v
GET /live  (Streamlit polls every 3 seconds)
                     |
                     v
Live Stream Page re-renders:
  - Status bar     (latest action + risk + latency)
  - KPI cards      (total predictions, anomalies, normal, avg RUL, avg risk)
  - Anomaly chart  (score vs prediction sequence, colour-coded)
  - RUL chart      (predicted RUL line over stream)
  - Risk chart     (area chart with 0.4 and 0.7 threshold lines)
  - Actions chart  (bar chart of recommended action distribution)
  - Feed table     (last 20 predictions, newest first, with timestamps)
```

7.15 Prometheus and Grafana Monitoring (monitoring/)

Prometheus is a time series monitoring system that collects metrics from the FastAPI /metrics endpoint at five second intervals and stores them with timestamps for historical analysis. Grafana connects to Prometheus as a data source and renders the metrics as charts, gauges, and alert panels. Together they provide the operational visibility that an engineering team needs to trust and oversee a deployed AI system over extended periods. The metrics tracked include the total prediction request rate, prediction failure rate, anomaly detection rate, API response latency at the fiftieth, ninety fifth, and ninety ninth percentiles, memory usage, CPU usage, open file descriptor count, and Python garbage collection health.

Step by Step Grafana Dashboard Setup

After running docker compose up --build, Grafana is available at http://localhost:3000 with the default credentials admin and admin. The following steps create a complete operational monitoring dashboard. These steps are performed once after the first deployment and the dashboard is then saved permanently for all future sessions.

8.1 Add Prometheus as a Data Source

Step 1 — Open Grafana

Navigate to http://localhost:3000 in your browser.

Log in with username admin and password admin. Skip the password change prompt for a local setup.

Step 2 — Open data source configuration

Click the gear icon in the left sidebar to open Settings.

Select Data Sources and then click Add data source.

Step 3 — Configure Prometheus connection

Select Prometheus from the list of available data sources.

In the URL field enter exactly: http://prometheus:9090

Leave all other settings at their defaults.

Scroll down and click Save and Test. A green confirmation message confirms the connection is working.

8.2 Create a New Dashboard and Add Panels

Step 4 — Create the dashboard

Click the Dashboards icon in the left sidebar.

Select New and then New Dashboard.

Click Add visualization to open the panel editor for the first panel.

Step 5 — Panel 1 — Total Prediction Requests

Set the visualization type to Stat.

Set the panel title to: Total Prediction Requests

In the query editor enter: telemetry_prediction_requests_total

Set the Unit to short. Click Apply.

Step 6 — Panel 2 — Prediction Failure Rate

Add a new Stat panel titled: Prediction Failure Rate

Enter the query: 100 * telemetry_prediction_failures_total / telemetry_prediction_requests_total

Set Unit to Percent (0-100). Under Thresholds set green from 0 and red from 1.

Click Apply.

Step 7 — Panel 3 — Anomaly Detection Rate

Add a Stat panel titled: Anomaly Detection Rate

Enter the query: 100 * telemetry_anomalies_detected_total / telemetry_prediction_requests_total

Set Unit to Percent (0-100). Set Thresholds: green at 0, orange at 10, red at 25.

Click Apply.

Step 8 — Panel 4 — Average Prediction Latency

Add a Stat panel titled: Average Prediction Latency

Enter the query: rate(telemetry_prediction_latency_seconds_sum[5m]) / rate(telemetry_prediction_latency_seconds_count[5m])

Set Unit to seconds (s). Set Thresholds: green at 0, orange at 0.5, red at 2.0.

Click Apply.

Step 9 — Panel 5 — Latency Percentile Time Series

Add a Time series panel titled: Prediction Latency Percentiles

Add three queries —

Query A label P50: histogram_quantile(0.50, rate(telemetry_prediction_latency_seconds_bucket[5m]))

Query B label P95: histogram_quantile(0.95, rate(telemetry_prediction_latency_seconds_bucket[5m]))

Query C label P99: histogram_quantile(0.99, rate(telemetry_prediction_latency_seconds_bucket[5m]))

Set Unit to seconds (s). Click Apply.

Step 10 — Panel 6 — API Memory Usage

Add a Time series panel titled: API Memory Usage

Enter the query: process_resident_memory_bytes

Set Unit to bytes (IEC). Set Thresholds: orange at 600000000, red at 800000000.

Click Apply.

Step 11 — Panel 7 — CPU Usage Rate

Add a Time series panel titled: CPU Usage Rate

Enter the query: rate(process_cpu_seconds_total[5m])

Set Unit to short. Click Apply.

Step 12 — Panel 8 — Memory Leak Indicator

Add a Stat panel titled: Uncollectable Objects (Memory Leak Check)

Enter the query: python_gc_objects_uncollectable_total

Set Unit to short. Set Thresholds: green at 0, red at 1.

Click Apply.

Step 13 — Panel 9 — Anomalies Over Time

Add a Time series panel titled: Anomalies Detected Over Time

Enter the query: rate(telemetry_anomalies_detected_total[5m])

Set Unit to short. Click Apply.

8.3 Arrange and Save

Step 14 — Arrange the panels

Row 1: Total Requests, Failure Rate, Anomaly Rate three stat panels side by side.

Row 2: Average Latency, Memory Leak Check two stat panels.

Row 3: Latency Percentile Chart full width time series.

Row 4: Memory Usage, CPU Rate two time series panels.

Row 5: Anomalies Over Time full width time series.

Step 15 — Save and enable auto refresh

Click the save icon at the top right.

Name the dashboard: Space Telemetry API Monitoring and click Save.

Set the refresh interval to 5s or 10s using the dropdown at the top right.

The dashboard will now update automatically as new predictions flow through the API.

Advantages of This Project

This project brings together a combination of technical depth, operational readiness, and mission relevance that distinguishes it from a standard machine learning exercise. The advantages are substantial and interconnected.

9.1 Fully Autonomous Operation with No Human Dependency

From the moment sensor data arrives to the moment a recommendation is issued, no human action is required. The system detects anomalies, predicts failure, makes decisions, quantifies risk, and delivers the output entirely on its own. This autonomy is the single most important property for any deep space mission where the communication delay makes ground based intervention too slow to be effective. The system can detect a critical fault and generate a corrective recommendation faster than any Earth based operator could even receive the initial alert.

9.2 Multi Model Redundancy

Rather than relying on a single model whose failure would compromise the entire decision chain, the system deploys two anomaly detectors and two RUL predictors based on fundamentally different mathematical principles. The Isolation Forest and the Autoencoder approach anomaly detection from completely different angles. The Random Forest and the LSTM use different learning strategies and different representations of time. When both models in a pair agree, confidence is high. When they diverge, the system signals low confidence so the output is treated with appropriate caution. This redundancy is a direct application of the fail safe engineering philosophy used in all critical aerospace systems.

9.3 Transparent and Fully Explainable Decisions

Every recommendation produced by the system is accompanied by a plain English justification identifying the exact measurements that drove the decision. Rather than a black box output that an

engineer must trust blindly, the system produces statements such as anomaly detected with a score of negative 0.18; RUL critically low at four cycles. This explainability is essential for building trust in an autonomous AI system, for conducting post mission analysis, and for certifying the system for use in a real aerospace programme where regulatory accountability requires that every automated decision can be fully audited.

9.4 Self Improving Policy through Reinforcement Learning

The rule based decision engine covers every scenario that was anticipated during design. The reinforcement learning agent extends this by discovering policies for scenarios that were not explicitly programmed. Given sufficient training episodes in a simulated environment, the agent learns to balance the operational cost of unnecessary shutdowns against the safety risk of ignoring developing faults, arriving at a nuanced policy that evolves with experience. This self improvement capability means the system can be adapted to new mission profiles and previously unseen failure modes without a complete redesign of the decision logic.

9.5 Production Grade Containerised Deployment

The system is not a research notebook. It is a fully containerised application where each of eight services runs in its own isolated Docker container with its own declared dependencies, starting in the correct order with a single command. This portability means the same deployment package that runs on a developer's laptop can be adapted for a server or an edge computing device on a spacecraft with minimal modifications. Containerisation also ensures reproducibility: the same code, the same library versions, and the same configuration will produce the same results on any machine that runs Docker.

9.6 Real Time Operational Self Monitoring

The integration of Prometheus and Grafana means the system continuously monitors its own health. Every prediction request is counted, every failure is logged, and every latency value is recorded and displayed on a live dashboard. Engineers watching Grafana can see in real time whether the API is healthy, whether anomaly rates are spiking unexpectedly, and whether resource usage is growing in a way that suggests a problem. This self monitoring capability is what distinguishes a trustworthy production system from a model that runs without feedback or visibility.

9.7 Real World Validated on an Established Benchmark

Using the NASA CMAPSS dataset rather than a purely synthetic alternative means the models have been trained and evaluated on realistic degradation patterns generated by a high fidelity NASA physics simulation. The results produced on this dataset are directly comparable to published literature, providing external validation that the models perform at a competitive level and that the system is not simply fitting noise. This external comparability strengthens the case for transferring the architecture to real mission data.

9.8 Streaming Architecture Ready for Live Telemetry

The Kafka pipeline demonstrates that the system can ingest and process a continuous real time data stream rather than only batch files. In a real mission, spacecraft telemetry arrives as a continuous feed. The producer consumer architecture shows that replacing the CSV reading producer with a live

telemetry receiver changes only the producer, leaving the entire downstream pipeline from Kafka through the API to the dashboard and monitoring stack completely untouched. This architectural separation of concerns is what makes the system genuinely extensible rather than tightly coupled to its current data source.

9.9 Live Dashboard Integration with Kafka Streaming

Unlike conventional streaming systems where the dashboard reads from a static file or requires a separate database, this system connects the Kafka pipeline directly to the dashboard through the FastAPI in memory live store. Every prediction made by the API triggered by the Kafka consumer is immediately visible on the Live Stream page of the dashboard, which polls the /live endpoint every three seconds and re renders all charts in real time. This means an engineer watching the dashboard can see the exact moment an anomaly is detected in the live stream, observe the recommended action change from NORMAL to WARNING or CRITICAL, and watch the risk score climb all without any manual refresh or file reload. The live feed table shows each prediction with its UTC timestamp and API latency in milliseconds, providing a complete audit trail of the streaming session.

Conclusion

This project demonstrates that a complete, end to end autonomous decision support system for spacecraft health management is achievable with current machine learning techniques and modern software engineering tools. Beginning with a raw stream of 21 sensor readings per operational cycle and finishing with a live mission control dashboard, a real time REST API, a streaming Kafka pipeline, and a Prometheus and Grafana monitoring stack, the system covers every stage of the journey from raw data to actionable decision.

The fundamental challenge addressed that deep space communication delays make ground based fault response too slow for real time emergencies is genuine and will only grow more pressing as India's space programme extends its reach from the Moon to the Lagrange points and eventually to Mars. The architecture demonstrated here provides a working blueprint for the kind of on board health management capability that future Chandrayaan, Aditya, and Gaganyaan missions will require.

The combination of multi model anomaly detection, sequence aware RUL forecasting, three tier autonomous decision making with risk quantification and confidence assessment, a self improving reinforcement learning agent, live Kafka to dashboard streaming integration, and full production grade containerised deployment makes this more than a collection of models it is an integrated, deployable system that functions as a whole. With the NASA dataset replaced by real spacecraft telemetry and the decision thresholds calibrated to the specific failure modes of a target mission, this system could be trusted today to make sound, autonomous maintenance decisions without waiting for guidance from Earth.

End of Document